An interesting point about the `Startup` class is that it doesn't implement an interface as such. Instead, the methods are invoked by using reflection to find methods with the predefined names of `Configure` and `ConfigureServices`. This makes the class more flexible and enables you to modify the signature of the `Configure` method to inject any services you registered in `ConfigureServices` using DI.

TIP If you're not a fan of the flexible reflection approach, you can implement the `IStartup` interface or derive from the `StartupBase` class, which provide the method signatures shown previously in listing 30.2. If you take this approach, you won't be able to use DI to inject services into the `Configure()` method.

`ConfigureServices` is where you add all your required and custom services to the DI container, exactly as you do with `WebApplicationBuilder.Services` in a typical .NET 7 ASP.NET Core app. The following listing shows how you might configure all the services for the Razor Pages recipe app you've seen throughout this book. This listing also shows how you can access the `IConfiguration` for your app: by injecting into the `Startup` constructor. You'll see how to customize your app's configuration in section 30.4.

Listing 30.3 Registering services with DI in `ConfigureServices`

```
public class Startup
{
    public IConfiguration Configuration { get; }                    ❶
    public Startup(IConfiguration configuration)                    ❶
    {
        Configuration = configuration;
    }

    public void ConfigureServices(IServiceCollection services)      ❷
```